

Cloud Computing
Semester 2026-III
Delivery No. 2 — Software Architecture

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the second delivery of the *Cloud Computing* course project!

In **Delivery 1**, your team defined the business problem, identified all actors and use cases, specified functional and non-functional requirements, and produced a quantitative workload model. You now have a precise answer to “*What does the system need to do?*” It is time to answer the next question: **How should the application be structured to satisfy those requirements?**

This delivery is entirely cloud-independent. You are designing the software — the services, their responsibilities, the data they own, how they communicate, and the key architectural decisions behind those choices. The cloud provider does not appear until Delivery 3. A strong software architecture is the foundation that makes a cloud architecture meaningful.

General Delivery Definition: Software architecture design involves decomposing a complex system into well-defined, independently deployable services, specifying the data architecture that supports those services, and defining the communication patterns that connect them. In this delivery, teams produce a complete, cloud-independent software architecture grounded in the requirements established in Delivery 1, supported by formal Architecture Decision Records that justify every significant design choice.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

Delivery Scope and Objectives

- **System Context Diagram:** Define who and what interacts with the system from the outside, establishing the external boundary of the architecture.
- **Service-Oriented Architecture:** Decompose the system into well-defined services with clear responsibilities, APIs, data ownership, and scaling characteristics.
- **Data Architecture:** Define the data model, storage strategy, consistency requirements, and transaction boundaries for each service.
- **Communication Architecture:** Specify synchronous and asynchronous communication patterns between services, with justification for each choice.
- **Architecture Decision Records:** Document the most significant design decisions with alternatives considered, rationale, trade-offs, and consequences.

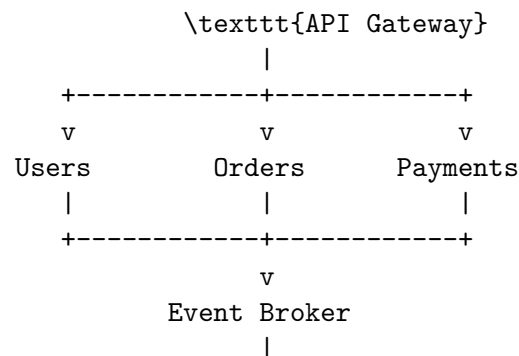
Required Artifacts and Methodology

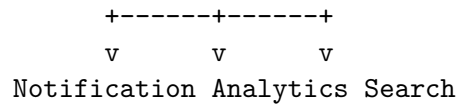
1. System Context Diagram:

- Produce a system context diagram (C4 Level 1) showing the system as a single black box and all external actors, users, and systems that interact with it
- For each external element, describe: what it is, how it interacts with the system, and at what frequency
- The diagram must be consistent with the actor and use case catalog from Delivery 1
- Label all interaction arrows with the nature of the communication (e.g., “submits ride request”, “sends payment confirmation”)

2. Software Architecture — Service Decomposition:

Decompose the system into independently deployable services. Present the architecture as a diagram showing services, their groupings, and their interaction topology. Example structure:





For each service in the architecture, define:

- *Responsibility*: What this service is solely responsible for; what it does and does not do
- *APIs*: Public interfaces exposed by the service (REST endpoints, gRPC methods, or event topics published); include HTTP method, path, and brief description for each
- *Data Owned*: What data this service owns exclusively; no other service may write directly to this service's data store
- *Dependencies*: Which other services this service depends on, and for what purpose
- *Scaling Characteristics*: Is this service stateless or stateful? What drives its scaling (CPU, memory, request volume, queue depth)? What are its expected instance counts at average and peak load?

3. Data Architecture:

- For each service, define its data storage strategy and justify the choice:
 - *Storage type*: Relational (RDBMS), document store, key-value, time-series, graph, or object storage — and *why*
 - *Data model*: Key entities and their relationships (ERD or schema sketch)
 - *Consistency requirements*: Strong consistency, eventual consistency, or causal consistency — and *why*
 - *Transaction boundaries*: Which operations require ACID transactions? How are distributed transactions handled across service boundaries?
- Define the overall data ownership model: which service owns which data, and how cross-service data access is handled (API calls, event subscriptions, or read replicas)
- Identify any shared data anti-patterns and explain how they are avoided in the design

4. Communication Architecture:

For each significant inter-service interaction, define the communication pattern and justify the choice:

- *Synchronous (REST/gRPC)*: Use when the caller requires an immediate response, the operation is short-lived, and failure must be surfaced immediately. Document: caller, callee, operation, and latency expectation.

- *Asynchronous (message queue / event bus)*: Use when the caller does not need an immediate response, the operation is long-running or non-critical, or when decoupling producers from consumers is required. Document: producer, event type, consumers, and delivery guarantee (at-least-once, exactly-once).
- Produce a communication matrix or diagram showing all inter-service interactions, their pattern (sync/async), and protocol
- Justify every asynchronous design choice: what would break if this were synchronous?

5. Architecture Decision Records (ADRs):

Document at least **five** Architecture Decision Records for the most significant design decisions in your architecture. Each ADR must follow this structure:

- *Decision*: What was decided? State the decision clearly in one sentence.
- *Context*: What requirement or constraint drove this decision? Reference specific requirements from Delivery 1.
- *Alternatives Considered*: What other options were evaluated? Describe at least two alternatives.
- *Rationale*: Why was this option chosen over the alternatives?
- *Trade-offs*: What does this decision give up? What are the downsides?
- *Consequences*: What does this decision imply for future design choices? What becomes easier or harder?

Suggested ADR topics for high-traffic applications:

- Monolith vs. microservices decomposition strategy
- Event-driven vs. request-driven communication for a core workflow
- Relational vs. NoSQL for a key data domain
- Synchronous vs. asynchronous payment processing
- CQRS pattern for read-heavy services
- Saga pattern vs. distributed transactions for multi-service operations

Deliverables and Documentation

1. Software Architecture Document (PDF):

- *Executive Summary*: Overview of the architecture approach and key design principles applied
- *System Context Diagram*: C4 Level 1 diagram with all external actors and interactions
- *Architecture Diagram*: Service decomposition diagram showing all services, groupings, and interaction topology

- *Service Catalog*: Complete definition of each service (responsibility, APIs, data owned, dependencies, scaling characteristics)
- *Data Architecture*: Storage strategy per service, data models, consistency requirements, and transaction boundaries
- *Communication Architecture*: Communication matrix or diagram with sync/async justification for each interaction
- *Architecture Decision Records*: Minimum five ADRs covering the most significant design decisions
- *Requirements Traceability*: Table mapping each functional requirement (FR-XX) from Delivery 1 to the service(s) responsible for satisfying it

2. Repository Update:

- Add the Software Architecture Document PDF to the `delivery_2/` folder
- Include all architecture diagrams as source files (`draw.io`, `PlantUML`, or equivalent) in `delivery_2/diagrams/`
- Update the `README.md` to reflect the current project status and link to both delivery documents

Submission Requirements

- The PDF report file must follow the naming convention: `groupN_delivery2.pdf`, where N is the team number (e.g., `group1_delivery2.pdf`, `group3_delivery2.pdf`)
- Submit the complete Software Architecture Document as a single PDF through the designated course platform
- Include the GitHub repository link with all diagrams and source files
- All documentation must be in **English** and follow professional technical writing standards
- **No cloud provider, managed service, or vendor-specific technology may be mentioned in this delivery** — the architecture must be completely cloud-independent
- All ADRs must reference specific requirements from Delivery 1 as their driving context

Important Guidelines and Considerations

- **Cloud Independence is Mandatory:** Do not mention AWS, GCP, Azure, DynamoDB, S3, SQS, or any cloud-specific service. Use generic terms: “relational database”, “message queue”, “object storage”, “API gateway”. The cloud translation happens in Delivery 3.
- **Service Boundaries Must Be Justified:** Every service boundary is a design decision. If you split a domain into two services, explain why. If you keep two related domains in one service, explain why. Use the ADRs for this.
- **Data Ownership is Non-Negotiable:** Each piece of data must have exactly one owning service. If two services need the same data, one must own it and the other must access it via API or event subscription. Shared databases between services are an anti-pattern.
- **Communication Choices Have Consequences:** Asynchronous communication introduces eventual consistency and complexity. Synchronous communication introduces coupling and latency sensitivity. Every choice must be justified against the requirements from Delivery 1.
- **Traceability to Requirements:** Every major design decision should trace back to a specific requirement. If a design element cannot be traced to a requirement, question whether it is necessary.
- **Prepare for Delivery 3:** Your software architecture will be directly translated into cloud infrastructure in Delivery 3. Design with deployability in mind: stateless services scale horizontally; stateful services require careful data management; event-driven services map naturally to managed queues.

Good luck, and embrace the challenge of designing a system before touching any technology — the discipline of thinking architecturally before acting operationally is what separates systems architects from implementors!